

KittyPatcher

v0.7.6 General Guide

Course of Temptation | SugarCube / Twine

Introduction

KittyPatcher is a Python-based patcher for the Twine/SugarCube game Course of Temptation (COT). It reads .mod files from a mods/ folder and applies their directives to the game HTML before launch. v0.7.0 added a dependency graph load-order system, two new guard clauses (IfPassageContains, IfPassageHasTag), a new Hook around: timing variant, TOML configuration, patch result caching, and a more reliable HTML processing layer via BeautifulSoup 4.

v0.7.4 adds: a content-addressed SHA-256 patch cache (startup time ~4 min to ~0.2 s on cache hit); a batch pre-pass for Append To Passage and Insert After/Before that eliminates repeated full-HTML scans; a two-pass anchor search so Replace: works on JS script-block targets (setup.storyhints.db entries etc.); and auto-close after browser launch. Three bugs fixed: cache not saved on failure, duplicate Hook conflict handling, and cache key including log files.

How it works: each .mod file contains one or more directives. KittyPatcher loads every .mod file it finds, applies all directives in a fixed seven-phase order, writes the patched game to CourseOfTemptation.patched.html, and opens the game in the browser. The original HTML is never modified -- a backup is taken on first run and restored at the start of every subsequent run.

Setup and File Layout

Place KittyPatcher.py (or the .exe) in the same folder as CourseOfTemptation.html, or in a parent folder up to two levels above it. The patcher auto-locates the HTML file on startup. On first run the patcher prompts you to select a browser; your choice is saved to kitty_config.toml.

Folder Structure

```
CourseOfTemptation.html
KittyPatcher.py          <- or KittyPatcher.exe
```

```

kitty_config.toml      <- browser + logging config
cm_dependencies.json   <- v0.7.0: optional mod load-order graph
mods/
  YourMod.mod
  AnotherMod.mod
logs/
  ModPatchLog.txt      <- every attempt, success, skip, failure
  FailsPatchLog.txt    <- failures and conflict warnings
  MainPatchLog.txt     <- summary: counts, timing, mods
  CustomLog.txt        <- mod-specific custom messages
backup/
  CourseOfTemptation.html <- auto-created on first run
cache/                 <- v0.7.4: content-addressed patch cache

```

NOTE: Pass the HTML path as a command-line argument for non-standard locations: `python KittyPatcher.py /path/to/CourseOfTemptation.html`

NOTE: v0.7.4: the patcher exits automatically after opening the game. No "Press Enter to exit" prompt on success.

v0.7.0: Dependency Graph and Load Order

In v0.6.0 and earlier, mods were applied in filesystem order. In v0.7.0, if `cm_dependencies.json` is present, the patcher builds a dependency graph and applies mods in topological sort order -- parents before children. Within the same level, mods are sorted by Priority: header field (default 1000, lower = earlier).

```

{
  "CM_ModB.v1": ["CM_ModA.v1"],
  "CM_SubmodAB.v1": ["CM_ModA.v1", "CM_ModB.v1"]
}

```

WARNING: If `cm_dependencies.json` contains a cycle (A requires B requires A), the patcher detects it, logs a cycle error to `FailsPatchLog.txt`, and falls back to Priority:/filesystem order. It does not abort the patch run.

Directive Overview

Every directive in a `.mod` file begins at column 0 (no leading spaces). v0.7.0 adds 2 new guards and the Hook around: timing variant. All v0.6.0 directives remain fully supported.

Directive	Purpose	New in v0.7.x?
Replace: / With:	Find and replace a block of text in the game HTML	
ReplaceReg [pattern]:	Regex-based find and replace with group targeting	
Add Passage:	Add a new passage (full or shorthand format)	
Add Javascript:	Append JavaScript to the game script block	
Add CSS:	Append CSS to the game style block	

Add Events:	Append entries to the events database array	
Directive	Purpose	New in v0.7.x?
Add Function [name]:	Inject a new named JS function; skipped if already exists	
Add Variable [name]: value	Inject a variable into StoryInit; skipped if present	
Replace Function [name]:	Replace the body of an existing JS function	
Replace Function Signature [name]:	Replace only the parameter list of a function	
Replace In Function [name]:	Replace anchor text within a named function body	
Replace In Passage [name]:	Replace anchor text within a named passage body	
Insert Before [target]:	Insert code immediately before a string target	
Insert After [target]:	Insert code immediately after a string target	
Insert Into Function [name]:	Insert code after an anchor within a named function	
Prepend To Passage [name]:	Insert content at the very start of a passage body	
Append To Passage [name]:	Insert content at the very end of a passage body	
Add Tag To Passage [name]: tag	Add tags to a passage tag attribute	
Remove Tag From Passage [name]: tag	Remove tags from a passage tag attribute	
Delete Block [name]:	Remove an entire named passage from the HTML	
Delete Span [start] To [end]:	Delete everything between two anchor strings	
Delete In Passage [name]:	Delete specific text within a named passage body	
Rename Passage [old]: new	Rename a passage name attribute	
Move Passage [name] To [target]:	Rename a passage and update all references	
Hook [name]:	Wrap an existing JS function (before/after/around)	around: is v0.7.0
Append Array [name]:	Append items to a named JS array	
Append Variable to Class [name]:	Add instance variables to a constructor	
Append Function to Class [name]:	Add methods to a class or prototype	
Insert Into Array [name]:	Insert items into a named array	
Insert Into Object [name]:	Insert key-value pairs into a named object	

KittyValidate: Pre-Flight Anchor Check

KittyValidate.py checks every directive anchor against the live game HTML without patching anything. Run it before applying a mod to catch anchors that will fail.

```
python KittyValidate.py CourseOfTemptation.html MyMod.mod
python KittyValidate.py --verbose CourseOfTemptation.html mods/
python KittyValidate.py --write-report CourseOfTemptation.html MyMod.mod
```

Status	Meaning
safe	Anchor found exactly once -- will apply correctly
ambiguous	Anchor found 2+ times -- matches first occurrence only; verify location
will fail	Anchor not found -- directive will error at patch time
would be added	Add Passage / Add Function target does not exist yet (correct)
already exists -- will be skipped	Add Passage / Add Function target already present
ST_UNSCOPED (v0.7.0)	Global anchor found but Insert Into Function would be safer

WARNING: The "ambiguous" status does not stop patching -- it is advisory. But an ambiguous Insert After that matches the wrong passage will inject code in the wrong location. Always resolve ambiguous before shipping a mod.

NOTE: mod_input can be a single file or a directory. Exits code 0 if zero failures, 1 if any fail.

Replace: / With:

The core directive. Finds an exact block of text in the game HTML and replaces it with new content. The search text must match the game HTML exactly, including whitespace and indentation. The patcher uses flexible newline matching at line boundaries but the content itself must be verbatim.

Basic Syntax

```
Replace:
<<set $gangbangChance to 0.05>>
With:
<<set $gangbangChance to 0.1>>
```

WARNING: Never use <e>...</e> inside the Replace: block itself. Only use it in With: bodies. The Replace: anchor must match the raw HTML encoding exactly.

The <e> Tag

Inside the game HTML, Twine macros are stored HTML-encoded. The <e>...</e> tag lets you write raw Twine syntax in With: bodies. In v0.7.0, With: bodies are auto-escaped so the <e> tag is

optional but still supported for backward compatibility.

```
Replace:
<<unset _map>>
<</if>>
With:
<e>
<<unset _map>>
<<unset _node>>
<</if>>
</e>
```

The [to] Marker: Wildcard Fill

Use [to] inside a Replace: block to automatically capture everything between a prefix and suffix. The patcher fills in whatever is between them and uses the complete result as the search target. Useful for targeting version-stamped values without knowing the exact current content.

```
Replace:
<<set $gangbangChance to [to]>>
With:
<<set $gangbangChance to 0.1>>
```

NOTE: Only one [to] per Replace: block. If no match is found the block is skipped gracefully.

Conflict Detection and Duplicate Check

When two mods target the same code with Replace:, a conflict is logged to MainPatchLog.txt and FailsPatchLog.txt. Both are applied in load order; the last applied wins. Duplicate patch detection probes unique content from the With: body and skips re-injection if already applied.

```
CONFLICT DETECTED: Replace: target found in multiple mods:
mod_a.mod -> same anchor as mod_b.mod
```

Add Passage:

Adds a new passage to the game. If a passage with the same name already exists it is skipped automatically and logged. Two formats are supported:

Full Format

```
Add Passage:
<tw-passagedata pid="auto" name="MyPassage" tags="event" position="0,0" size="100,100">
<<set $myVar to 1>>
Hello from my passage!
</tw-passagedata>
```

Use pid="auto" and the patcher assigns a correct sequential PID during renormalization. The patcher auto-escapes raw Twine macros inside the passage body. Duplicate passage names across mods are logged to FailsPatchLog.txt; the second copy is skipped silently.

Shorthand Format

```
Add Passage: MyPassage
```

```
tags: event nobr
<<set $myVar to 1>>
Hello from my passage!
```

The tags: line is optional. Body ends at the next directive token or end of file. Add Passage blocks are extracted into a passage registry during parsing and injected in Phase 7 after all other directives have run.

WARNING: Add Passage blocks are NOT available to Replace Function during Phase 4. They are injected in Phase 7. You cannot Replace Function a function that lives in an Add Passage block in the same load.

Add Javascript: / Add CSS: / Add Events: / Add Function: / Add Variable:

Add Javascript:

Appends JavaScript code to the end of the main script block. Multiple blocks accumulate in order. Injected in Phase 7. Preferred for config objects, event listeners, or prototype extensions.

```
Add Javascript:
Person.prototype.ownedresidences = [];
Person.prototype.residencesbadcredit = 0;
```

WARNING: Add Javascript: has no duplicate check. Wrap stateful setup code in a guard: if (setup.myConfig) { ... }. Running the patcher twice without this guard will double-execute the setup code on page load.

Add CSS:

Appends CSS rules to the end of the style block. /* */ block comments are safe in CSS. DO NOT use // line comments in CSS -- they are not valid CSS syntax and silently break rules following them on that line.

Add Events:

Appends entries to the game events database array (setup.Events.db). The trailing comma on the last entry is required. Duplicate event passage registrations cause the event to fire ~Nx as often.

```
Add Events:
{
  passage: "MyEventPassage",
  tags: ["dorm", "gym"],
  chance: 0.05,
  conditions: [],
},
```

Add Function [name]:

Injects a new named JavaScript function into the script block. Skipped if a function with that name already exists. Permanently version-proof for the function logic itself. Supports assignment-style, declaration-style, and method shorthand.

```
Add Function [setup.myNewFunction]:
setup.myNewFunction = function(arg) {
    return arg * 2;
};
```

NOTE: Within Phase 4, Add Function entries are sorted before Insert Into Function entries. You CAN Insert Into Function a function that the same mod defines via Add Function.

Add Variable [name]: value

Injects <<set \$varname to value>>> into StoryInit. Skipped if the variable name already appears in StoryInit. Tier S resilience -- anchors on StoryInit name and variable name.

```
Add Variable [myModEnabled]: true
Add Variable [myModScore]: 0
```

Replace Function / Signature / In Function

Replace Function [name]:

Replaces the entire body of an existing JavaScript function. The function signature is preserved; only the inner body is replaced. Write only the body contents -- not the surrounding braces.

```
Replace Function [setup.someExistingFunction]:
// New body
return doSomethingDifferent();
```

WARNING: You own the ENTIRE function body. Dev changes to that function in future updates are silently discarded. When two mods Replace Function the same target, a conflict is logged and the last applied wins. Prefer Hook + Add Function whenever possible.

Replace Function Signature [name]: newParams

Replaces only the parameter list of a named function, leaving the body completely intact. Useful when a mod needs to add new parameters without owning the full function body.

```
Replace Function Signature [setup.random_name]: name, ethnicity, traitassocid
```

Replace In Function [name]:

Scoped replacement that replaces anchor text only within the named function body. The anchor only needs to be unique within that function, not the entire HTML. Prevents false matches when the same code snippet appears in multiple functions throughout the HTML.

```
Replace In Function [setup.someFunc]:
return oldValue;
With:
return newValue + bonus;
```

Insert Before / After / Into Function

Insert Before [target]: and Insert After [target]:

Finds the first occurrence of the target string in the entire HTML and injects the With: body immediately before or after it. The target is a literal string, not regex. A duplicate check skips re-injection if the With: body is already present immediately adjacent to the target.

```
Insert Before [setup.Events.db =]:  
With:  
// Events injected by MyMod
```

```
Insert After [Person.prototype.initState = function() {}]:  
With:  
this.modStat = 0;
```

WARNING: Insert After and Insert Before are global -- they match the FIRST occurrence of the target anywhere in the HTML. If the same anchor appears in multiple passages or functions, use Insert Into Function or Replace In Passage instead. KittyValidate v0.7.0 reports ST_UNSCOPED for these cases.

Insert Into Function [name]:

Scoped Insert After that only matches inside the named function body. Prevents false matches when the anchor string appears in multiple functions. The patcher locates the function by name using bracket-matching, finds the anchor within that body, and injects after it.

```
Insert Into Function [setup.build_encounter]:  
var result = {};  
With:  
result.modFlag = true;
```

NOTE: Within Phase 4, Add Function is sorted before Insert Into Function. You can Insert Into Function a function that the same mod defines via Add Function in the same .mod file.

Prepend / Append To Passage: / Add Tag / Remove Tag:

Prepend To Passage [name]: and Append To Passage [name]:

Injects the With: body at the very start (Prepend) or very end (Append) of a named passage body. Anchors on the passage name only -- no anchor string within the passage is needed. This is safer than Insert Before/After when the passage first or last line changes across game versions (StoryInit, PassageFooter, any frequently-modified passage).

```
Append To Passage [Cheats]:  
With:  
<<link "My Mod Debug">>  
<<goto "MyModDebug">>  
<</link>>
```

```
Prepend To Passage [StoryInit]:  
With:  
<<set $myModVersion to "1.0.0">>
```

Duplicate check: if the With: body is already present at the start/end, the block is skipped. When two Prepend directives target the same passage, both apply -- the second goes before the first.

Add Tag To Passage [name]: tag and Remove Tag From Passage [name]: tag

Modifies the tags="..." attribute of a named passage. Add Tag appends new tags if not already present. Remove Tag strips a specific tag. Tier S resilience -- anchors on passage name only. Eliminates fragile Replace: patterns targeting passage opening tags.

```
Add Tag To Passage [ShowerStall]: noevents
Remove Tag From Passage [MyPassage]: oldtag
```

Delete Directives

Delete Block [name]:

Removes an entire named passage from the game HTML. No duplicate check -- if the passage is already absent, the directive logs a failure but does not abort patching.

```
Delete Block [SomeUnwantedPassage]:
```

WARNING: Use Delete Block sparingly. If the dev keeps the passage but your mod deletes it, any game logic that depends on it will break silently. Always verify there are no remaining references to the passage name in vanilla.

Delete Span [start] To [end]:

Deletes everything from start_anchor through end_anchor, inclusive. No With: body needed. Both anchors must be present and start must precede end.

```
Delete Span [<<set _oldvar to true>>] To [<<unset _oldvar>>]:
```

Delete In Passage [name]:

Scoped deletion that removes anchor text from the named passage body only. Write raw Twine in the anchor -- the patcher encodes it.

```
Delete In Passage [ShowerStall]:
<<set _oldvar to true>>
```

Hook [name]: (before / after / around)

Wraps an existing JS function with custom code that runs before, after, or around the original. The original function is always preserved. v0.7.0 adds the around: timing variant.

before: timing

```
Hook [setup.people.generate]:
before:
console.log("About to generate an NPC");
```

after: timing

```
Hook [setup.people.generate]:
after:
console.log("Generated:", _result);
// _result holds the return value
```

around: timing (v0.7.0)

The around: variant gives full control over _result. Your code wraps the call and can inspect or replace the return value before it propagates. Must explicitly return a value.

```
Hook [setup.some_check]:
around:
var _result = setup_some_check__orig.apply(this,arguments);
if (_result && State.variables.myModActive) _result = _result * 1.5;
return _result;
```

WARNING: Only assignment-style functions (name = function(...) {}) are supported as hook targets. Hook is skipped if the wrapper variable (name __orig) already exists. _result is only available in after: and around: hooks -- do not reference it in before: hooks.

ReplaceReg [pattern]: and Structural Directives

ReplaceReg [pattern]:

Uses a Python regex pattern to find a match in the game HTML, then replaces one or more capture groups. Useful when surrounding context is variable but the pattern is stable.

```
ReplaceReg [(?s)somePrefix(GROUP_CONTENT)someSuffix]:
With{1}:
replacement for group 1
```

NOTE: Use (?s) to enable DOTALL mode (. matches newlines). Group 0 is the full match. Multiple With{N}: blocks in one directive replace different groups. Applied right-to-left.

Structural Directives

Append Array, Append Variable to Class, Append Function to Class, Insert Into Array, Insert Into Object. These modify named arrays and classes by anchoring on a name, giving them near-Tier S resilience.

```
Append Array [setup.hairLengths]:
"Very Long",
"Extremely Long",

Append Variable to Class [Person]:
this.pregnancyWeek = 0;

Append Function to Class [Person]:
getPregnancyStage() {
  if (this.pregnancyWeek<13) return "first";
  return "second";
}

Insert Into Array [setup.kinklist]:
```

```
"newKink",
```

```
Insert Into Object [setup.config]:  
myModKey: true,
```

NOTE: All structural directives support optional In Function scoping: Append Array [someArray In Function [setup.init]]:

Guard Clauses

Any directive block can include guard lines that make the patch conditional. Guards are evaluated against the current HTML at patch time. If any guard fails, the entire block is skipped and logged.

Guard	Condition
IfExists [value]:	Apply only if string exists anywhere in the HTML
IfNotExists [value]:	Apply only if string is absent from the HTML
IfRegExists [pattern]:	Apply only if regex matches the HTML
IfRegNotExists [pattern]:	Apply only if regex does not match the HTML
IfModLoaded [filename]:	Apply only if the named .mod file is loaded
IfModNotLoaded [filename]:	Apply only if the named .mod file is NOT loaded
IfPassageExists [name]:	Apply only if passage exists in HTML or registry
IfPassageNotExists [name]:	Apply only if passage does not exist
IfFunctionExists [name]:	Apply only if function exists in script block
IfFunctionNotExists [name]:	Apply only if function does not exist
IfPassageContains [Name text]: (v0.7.0)	Apply only if named passage body contains the text
IfPassageHasTag [Name tag]: (v0.7.0)	Apply only if named passage has the exact tag token

```
Replace:  
<<set $gangbangChance to 0.05>>  
With:  
IfExists [setup.Events.db]:  
IfNotExists [gangbangEnabled]:  
IfRegExists [setup\.people\.generate\s*=\s*function]:  
<<set $gangbangChance to 0.1>>
```

Guards are placed inside the With: body. Guards preceding a directive on a separate line are automatically hoisted into the correct body region. Multiple guards of the same type are allowed and all must pass. Guards are stripped from the With: body before injection -- they never appear in the patched HTML.

Processing Order (v0.7.0)

During parsing, directives are sorted into separate lists. During application, they execute in a fixed seven-phase order. v0.7.0 adds Phase 0 for dependency graph resolution before patching.

Phase	What runs	Details
0 (v0.7.0)	Dep graph sort	If cm_dependencies.json is present, mods are topologically sorted. Falls back to Priority:/filesystem order on missing file or cycle.
1	Replace: entries	Replace:/With: pairs in mod load order. Passage-targeting entries use name-based fallback with pid-aware diagnostics if exact match fails.
2	ReplaceReg entries	Regex-based replacements against the full HTML.
3	Structural entries	Append Array, Append Variable/Function to Class, Insert Into Array/Object.
4	Func/passage-scope d (sorted)	Add Function (sorted first), then Insert Into Function, Replace Function, Replace Function Signature, IA/IB, Delete Block/Span, RIP/RIF, DIP, Move/Rename, Tag ops, Prepend/Append To Passage, Add Variable.
5	Hook entries	Function wrapping with before/after/around hooks.
6	Failed Replace: sweep	Retry failed Phase 1 entries against passage registry.
7	Registry injection	(a) Add JS to script block, (b) Add CSS to style block, (c) Add Events to Events.db, (d) Add Passage before storydata close. PID renormalization runs after.

WARNING: Add Passage / Add Javascript / Add CSS / Add Events are injected in Phase 7 after all other directives have run. Replace Function (Phase 4) cannot target a function added by Add Function in the same load. Add Function is sorted before Insert Into Function within Phase 4.

PID Renormalization, Conflict Detection, and Logs

PID Renormalization

After all patches, the patcher rennumbers every <tw-passagedata pid="..."> sequentially (1, 2, 3...) in document order. SugarCube identifies passages by name, not pid, so this is completely safe. The startnode attribute in <tw-storydata> is updated. v0.7.0 uses BeautifulSoup 4 for more reliable PID handling on malformed or edge-case HTML.

Conflict Detection

When two or more mods target the same function or passage with directives that overwrite, the patcher logs a conflict warning to both MainPatchLog.txt and FailsPatchLog.txt after all patches complete.

Conflicts are reported but not blocked. Resolve by refactoring to Hook + Add Function.

Integrity Checks

After all mods are applied the patcher checks: `<tw-passagedata>` tag balance (open vs close count), and Twine macro balance (if, for, switch, widget, link, capture, highlight, known). Per-injection tag balance checks run on each affected passage. Warnings appear in the logs but do not stop the game from launching.

Log Files

File	Contents
ModPatchLog.txt	Every attempt, success, skip, and failure with the search key
FailsPatchLog.txt	Only failed replacements and conflict warnings
MainPatchLog.txt	Summary: total replacements made/failed, successful/failed mods, timing
CustomLog.txt	Custom messages from mod-specific logging

Verbose Logging

Enable with `--verbose` or `-v` on the command line, or by setting `verbose=true` in the `[logging]` section of `kitty_config.toml`.

```
python KittyPatcher.py --verbose
python KittyPatcher.py -v
# kitty_config.toml:
[logging]
verbose = true
```

Rename Passage / Move Passage in Depth

Rename Passage [OldName]: NewName

Updates the `name=` attribute on a named passage opening tag only. It does NOT update other passages that reference the old name via `<<goto>>>` or `<<display>>>`. If those references exist, use Move Passage instead.

```
Rename Passage [OldEventName]: NewEventName
```

Move Passage [OldName] To [NewName]:

Renames the passage AND updates all `<<goto>>>`, `<<link>>>`, `<<display>>>`, `<<include>>>`, and `passage:` references throughout the entire HTML. Tier S resilience -- zero porting cost.

```
Move Passage [EventTownWalkOld] To [EventTownWalkNew]:
```

WARNING: Rename Passage only changes the name attribute. If other passages use <<goto "OldName">> those references are NOT updated and will break at runtime. Always use Move Passage when the old passage name is referenced elsewhere in the game.

How Multiple Mods Interact

All .mod files in the mods/ folder are loaded and applied on every patch run. The patcher restores from the clean backup before applying mods, so the output is always a clean single-pass result regardless of how many times you run the patcher.

Load order in v0.7.0

If cm_dependencies.json is present, mods are loaded in topological sort order (parents before children). Within the same dependency level, mods are sorted by the Priority: field (default 1000, lower = earlier). Without cm_dependencies.json, mods load in filesystem order (typically alphabetical).

The correct approach: hooks and additive directives

When two mods need to modify the same function, neither should use Replace Function. Both should use Hook + Add Function. Each mod hooks the function independently, both execute without awareness of each other, and the vanilla function is always preserved.

Add Events: Entry Format Reference

The Add Events: directive appends entries to setup.Events.db. Each entry is a JavaScript object. A missing comma or incorrect field name will silently break event scheduling.

```
Add Events:
{
  passage: "EventMyModDorm",      // passage name (required)
  tags: ["dorm", "morning"],      // location/time tags that gate this event
  chance: 0.05,                  // base fire chance (0.0 to 1.0)
  conditions: [],                 // additional JS condition strings
},
```

WARNING: Duplicate event entries (same passage name from two mods) cause the event to fire at approximately 2x intended frequency. Always check for duplicate event registrations when combining mods.

Directive Stability Tiers: Practical Guide

Choosing the right directive is not just about capability -- it is about how much ongoing maintenance the directive creates. A mod that uses only Tier S and Tier 1 directives rarely if ever needs updating after a game release. A mod built on Replace: blocks can require updates after every release.

How to think about tier selection

For every change you want to make, ask: "Can I express this as Add Function + Hook?" If yes, do that. If no, try Insert After / Insert Before. If those don't work, try Replace In Passage or Insert Into Function. Reserve Replace: for cases where nothing else works.

Example: adding a call to a function

```
// FRAGILE (Replace: approach):
Replace:
setup.doThing(config, false);
With:
setup.doThing(config, false);
setup.myMod.track();

// RESILIENT (Hook approach):
Add Function [setup.myMod_track]:
setup.myMod_track = function() {
  /* your tracking logic */
};
Hook [setup.doThing]:
after:
  setup.myMod_track();
```

The Hook approach has zero porting cost. If setup.doThing changes signature, your Hook still works -- it just wraps whatever version exists.

v0.7.0 Structured Mod Header Reference

In v0.7.0, the patcher parses specific fields from the mod file header (# comment lines). These fields are optional but strongly recommended for any mod distributed to other users.

Field	Format	Purpose
Mod Name:	# Mod Name: CM_MyMod	Display name for logs and toolchain output
Version:	# Version: 1.0.0	Semantic version string for the mod
Author:	# Author: CoderMaverick	Mod author credit (used in release toolchain)
Requires:	# Requires: CM_ModA.v1, CM_ModB.v1	Comma-separated list of required mod files. Advisory only -- emits warning if absent.
Priority:	# Priority: 500	Integer. Lower = loads earlier within same dep level. Default 1000.
Description:	# Description: What this mod does	One-line description for logs and toolchain output

NOTE: The Requires: field does NOT enforce load ordering -- only cm_dependencies.json does. Requires: emits a warning to MainPatchLog.txt if a named mod file is not found in the mods folder, but does not block patching.

Escaping Reference: Complete Rules

Understanding the escaping context for every part of a .mod file prevents the most common class of silent failures.

HTML encoding in the game file

Twine stores passage content in the HTML as HTML-encoded text. The string `<<if $foo>>` in Twine source becomes `&lt;&lt;if $foo&gt;&gt;` in the HTML file. When writing Replace: anchors, you must use the HTML-encoded form because that is what the patcher searches for.

Script blocks are not HTML-encoded

JavaScript in the game's `<script>` block is NOT HTML-encoded. The string `<<if>>` does not appear in the script block -- only in passage content. When your Replace: target is inside a `<script>` block, write the JS syntax directly without HTML encoding.

The patcher auto-escape (v0.7.0)

In v0.7.0, With: bodies for passage content directives are automatically escaped before injection. You write `<<if $foo>>` in your With: body and the patcher converts it to the HTML-encoded form before inserting it into the passage body. The `<e>` tag is no longer needed for new mods.

Add Passage: bodies are always auto-escaped

Regardless of version, passage content in Add Passage: blocks is always auto-escaped. You write raw Twine in the passage body and the patcher converts it before injection. This has been true since v0.5.3.

Insert After anchor encoding

Insert After and Insert Before anchors are provided as decoded Twine in the .mod file. The patcher encodes them internally before searching the HTML. Exception: anchors inside `<script>` blocks use raw JS syntax because the script block is not HTML-encoded.

Location	What to write
Replace: block	HTML-escaped form: copy from game HTML directly
With: body (Twine passage)	Raw Twine in v0.7.0 (auto-escaped); use <code><e>...</e></code> for v0.6.0 compat
With: body (JS script)	Write raw JS -- patcher unescapes automatically for script context
Add Passage: body	Write raw Twine -- patcher auto-escapes passage bodies
Add Javascript: body	Write raw JS -- no escaping needed

KittyPatcher and the Ecosystem: What the Patcher Does Not Own

KittyPatcher is a pure HTML transformation tool. It has no awareness of which mods are CM-compatible, no enforcement of the CM namespace conventions, and no validation of game-logic correctness. The ecosystem conventions (namespace rules, save compatibility patterns, typeof guards, submod patterns) are the responsibility of mod authors, not the patcher.

The patcher will successfully apply a .mod file that writes into another mod's namespace, creates save-incompatible state, or skips typeof guards. It will not warn you. These are authoring errors that only show up at game runtime. KittyValidate can only catch anchor failures -- it cannot validate semantic correctness.

What the patcher guarantees

If a directive is applied, the injection appears in the patched HTML exactly as written.

If a directive fails, it is logged and skipped -- no partial injection.

The original game HTML is never modified -- the patched output is always a separate file.

PID renormalization runs after all patches, ensuring no PID collisions regardless of mod count.

Per-injection tag balance warnings flag structurally broken injections before the game launches.

What the patcher does not guarantee

Semantic correctness of injected code -- if your JS has a bug, the patcher injects the bug.

Namespace isolation -- writing into another mod's namespace is allowed at the patcher level.

Save compatibility -- ?? defaults are a mod authoring convention, not a patcher enforcement.

Cross-mod compatibility -- two mods that conflict in their game logic will both be applied.

Structural Directives: In Depth

The structural directives (Append Array, Append Variable to Class, Append Function to Class, Insert Into Array, Insert Into Object, Delete Block) share a common characteristic: they anchor on a name in the source code rather than on a specific line of text. This gives them near-Tier S resilience -- they survive most game updates unless the game renames the specific array, class, or object they target.

Append Array vs Insert Into Array

Both directives append content to a named array. The difference is positional: Append Array inserts content before the array's closing bracket (at the end of the array). Insert Into Array also inserts at the end of the array. In practice, both directives produce the same result for simple value arrays. For complex objects that need to be at a specific position in the array, neither directive gives positional control -- consider Replace: in that case.

Append Variable to Class: ES6 vs prototype

The patcher detects whether the target class uses ES6 class syntax or prototype-style constructor syntax and inserts variables in the correct location:

```
// ES6 class:  
Append Variable to Class [Person]:  
this.myModProp = 0;  
// Inserts inside constructor() {}
```

```
// Prototype style:  
Append Variable to Class [Person]:  
this.myModProp = 0;  
// Inserts at end of Person = function(...) { ... }
```

Append Function to Class [name]:

Appends a method to an existing class or prototype. For ES6 classes, the method is inserted before the closing brace of the class definition. For prototype classes, the method is inserted as a prototype assignment.

```
Append Function to Class [Person]:  
getModScore() {  
    return this.myModProp * 10;  
}
```

Log Files in Depth

ModPatchLog.txt

The most detailed log. Contains every directive attempt with its search key, the result (applied / skipped / failed), and for Replace: operations in verbose mode, the full old and new content. Always open this file first when a mod does not behave as expected.

FailsPatchLog.txt

Contains only failures and conflict warnings. If this file is empty after a patch run, all directives applied successfully. If non-empty, at least one directive failed or two mods conflicted. Check this file first for a quick health check.

MainPatchLog.txt

Summary log: total directives applied, total failures, list of successfully patched mods, list of failed mods, timing breakdown per phase.

WARNING: All logs are cleared at the start of every patch run. If you need to preserve a log for comparison, copy it before re-running.

Common Authoring Mistakes

Anchors that are too short or too common

A Replace: anchor like `<<set $count to 0>>` will match the FIRST occurrence of that exact string anywhere in the 16MB+ game HTML. If that string appears in 12 different passages, your mod will always patch the same first one. Keep anchors unique: use at least 2-3 lines of surrounding context, or switch to Replace In Passage to scope the match to the correct passage.

Duplicate event registrations

If two mods both add an event entry for the same passage name, the event fires twice as often. The patcher logs duplicate event registrations to FailsPatchLog.txt but does not block them. Always check FailsPatchLog.txt after every patch run.

Replace Function when Hook would do

Replace Function gives you the entire function body to own and maintain. If the game developer fixes a bug or adds a parameter in the next update, your mod silently discards those changes. Reserve Replace Function for cases where you genuinely need to restructure the entire function. For simple additions, use Hook + Add Function instead.

v0.7.4: Patch Result Cache

v0.7.4 introduces a content-addressed SHA-256 patch cache. On a cache hit, the full patch phase (approximately 4 minutes with a large mod set) is skipped entirely. The cached patched HTML is served directly, reducing startup time to approximately 0.2 seconds.

How the cache key is computed

The cache key is a SHA-256 hash of: (1) the vanilla game HTML file, and (2) every .mod file in the mods/ folder, concatenated in load order. Any change to any mod file or to the vanilla HTML invalidates the cache automatically. The logs/ subdirectory is excluded from the key computation so that log file changes do not cause spurious cache misses.

Cache configuration

```
[cache]
enabled = true      # set to false to force a full re-patch every run
max_entries = 10    # prune oldest entries beyond this count
```

NOTE: The cache is stored in the cache/ directory alongside the patcher. It is automatically pruned to max_entries entries, removing oldest first.

NOTE: The cache is NOT written when the patch run has failures. Only clean patch runs are cached.

v0.7.4: Batch Pre-Pass for Append and Insert

Before v0.7.4, every Append To Passage and Insert After/Before directive performed its own full-HTML regex scan independently. With a large mod set (207 Append To Passage calls across 82 mods), this caused 207 separate regex passes over the full 22MB game HTML.

v0.7.4 adds a batch pre-pass: all Append To Passage and Insert After/Before directives targeting the same passage or anchor are collected first, then applied together in a single right-to-left pass. The number of full-HTML regex scans drops from one per directive to one per unique target.

NOTE: The batch pre-pass is transparent to mod authors. No changes to .mod file syntax are required. Injection order is preserved.

v0.7.4: Replace: Two-Pass Anchor Search

Before v0.7.4, Replace: anchors were HTML-encoded before searching. This is correct for passage content (HTML-encoded in the game file), but wrong for JavaScript object entries in the `<script>` block. Script block content stores raw characters -- raw quote characters, not `"` entities.

The result: Replace: anchors targeting JS object entries in the script block (for example, entries in `setup.storyhints.db` or `setup.inclinations`) would silently fail with a "No match found" entry in `ModPatchLog.txt`, even though the anchor text was written correctly.

v0.7.4 fix: two-pass search

The patcher tries the encoded anchor first (correct for passage content). If the encoded search misses and the anchor contains no passage tag (indicating a script-block target), the raw unencoded anchor is tried. The log distinguishes fallback hits:

```
Replace: encoded anchor missed -- raw script-block fallback succeeded
for 'setup.storyhints.db entry...' (v0.7.4)
```

NOTE: Mod authors do not need to change anything. Write Replace: anchors targeting script-block content the same way you always have -- raw JS syntax, no HTML encoding.

IMPORTANT: For script-block object entries, prefer Insert Into Object [objectName]: over Replace: wherever possible. Insert Into Object is Tier S. Replace: on script-block entries is Tier 5 even with the v0.7.4 fix.

v0.7.4: Auto-Close After Browser Launch

Before v0.7.4, the patcher displayed a "Press Enter to exit" prompt after opening the game in the browser. The browser was already running independently -- the patcher process had nothing left to do but wait.

v0.7.4 removes this prompt for normal (non-watch) runs. After opening the browser, the patcher exits cleanly. The terminal window closes.

NOTE: Watch mode is unaffected -- `--watch` runs continue until manually interrupted. Error conditions are also unaffected: if the patch fails, the patcher still waits for input so you can read the error message.

v0.7.4: Hook Conflict Handling Fix

Before v0.7.4, when two mods both hooked the same function, the second hook was silently dropped. The conflict was logged, but only the first mod's hook body executed.

v0.7.4 fix: when a second (or third, or fourth) mod hooks the same function, the new hook body is appended into the existing wrapper. All mods fire in load order. The original function is called once. No hook body is lost.

```
// ModA.mod:
Hook [setup.build_encounter]:
```

```

after:
setup.modA_track();

// ModB.mod:
Hook [setup.build_encounter]:
after:
setup.modB_track();

// v0.7.4 generated wrapper (both fire):
setup.build_encounter = function() {
    var _result = setup_build_encounter__orig.apply(this, arguments);
    setup.modA_track();    // from ModA
    setup.modB_track();    // from ModB
    return _result;
};

```

Same-Mod Duplicate Directives

A mod file must never contain two directives targeting the same passage or function from within the same file. This is the most common source of conflict log noise and, for Hook directives, a functional bug.

Same-timing Hook duplicates are a functional bug

When the same mod has two Hook [setup.func]: around: directives, the second hook wraps the first hook's wrapper. The second call to _original() calls the first hook's wrapper, not the real function -- effectively double-applying the first hook's logic. Always merge same-timing Hook bodies into a single directive.

WARNING: Duplicate same-timing Hook directives within one mod file double-apply logic. Different-timing hooks (around: and after:) on the same function from the same mod are not duplicates -- they may remain separate.

Append To Passage duplicates

Multiple Append To Passage [X]: blocks in one mod file produce conflict log entries but do not break functionality -- all With: bodies are applied sequentially. Consolidate into a single directive for cleanliness.

```

// WRONG: two blocks for the same target
Append To Passage [EventCleanup]:
With:
<<run setup.mymod_cleanup(>>>

Append To Passage [EventCleanup]: // duplicate
With:
<<set V._cmModFlag to false>>

// CORRECT: one consolidated block
Append To Passage [EventCleanup]:
With:
<<run setup.mymod_cleanup(>>>

```

```
<<set V._cmModFlag to false>>
```

Surgical Updates: Extend, Never Duplicate

When updating an existing mod to add new content to a passage or function it already touches, always extend the existing directive rather than adding a second one. This is the most common source of same-mod duplicate conflicts.

```
// WRONG: v1.2.0 adds a second Append To Passage for the same target
// (v1.1.0 block already in the file):
Append To Passage [CMPPharmacy]:
With:
<<existing content from v1.1.0>>

// v1.2.0 update (WRONG -- creates a duplicate):
Append To Passage [CMPPharmacy]:
With:
<<new content from v1.2.0>>

// CORRECT: extend the existing block
Append To Passage [CMPPharmacy]:
With:
<<existing content from v1.1.0>>
<<new content from v1.2.0>>
```

BEST PRACTICE: Before adding any directive to a mod update, check whether the mod already has a directive for that target. If it does, extend the existing With: body. Never create a second directive for the same target.

Orphan Directives

An orphan directive is one that cannot fire correctly because its target does not exist. Common cases:

An Append To Passage that targets a passage no mod or vanilla game defines.

A Hook that targets a function not defined anywhere in the script block.

An Add Events: entry whose passage field names a passage that does not exist.

An Insert Into Function targeting a function nobody defines.

Orphan directives for hook surfaces (passages or functions that a parent mod provides for submods to extend) are valid when documented in the mod header's Provides: field. Undocumented orphans that can never fire are dead code.

NOTE: KittyValidate reports "will fail" for directives whose anchor is absent from the HTML. Run KittyValidate against every game version after a game update to find newly orphaned directives.

v0.7.4 New Features Summary

Feature	Details
---------	---------

Patch cache	Content-addressed SHA-256 cache. Cache hit skips full patch phase (~4 min to ~0.2 s). Configurable via [cache] in kitty_config.toml. Not written on failure. Log files excluded from key.
Batch pre-pass	Append To Passage and Insert After/Before grouped by target and applied in a single pass per target. Eliminates repeated full-HTML scans for large mod sets.
Two-pass search	Replace: Tries encoded anchor first (passage content), then raw anchor if miss and no passage tag (script-block content). Fixes silent failure on JS object targets.
Auto-close after launch	Terminal exits cleanly after opening the game. "Press Enter" prompt removed for normal runs. Watch mode and error conditions unaffected.
Hook conflict fix	Second+ mod hooks on the same function now append into the existing wrapper. All mods fire in load order.
Cache invalidation fix	Cache not written when patch has failures. Only clean runs cached.
Cache key fix	Log files in logs/ excluded from cache key hash. Log changes no cause spurious misses.

First-Time Setup: Step-by-Step

This section walks through the complete setup from a fresh download to a working modded game. Follow these steps in order.

Step 1: Place the patcher

Download KittyPatcher and place the files in one of these locations:

```
# Option A: same folder as the game
CourseOfTemptation.html
KittyPatcher.py          <- place here

# Option B: one folder above
KittyPatcher.py
GameFolder/
  CourseOfTemptation.html

# Option C: two folders above
KittyPatcher.py
Games/
  COT/
    CourseOfTemptation.html
```

NOTE: The patcher searches up to two parent levels automatically. You do not need to specify the HTML file path unless you use a non-standard layout.

Step 2: Create the mods folder

Create a folder named `mods/` in the same directory as `KittyPatcher.py`. This is where all your `.mod` files go.

```
KittyPatcher.py
mods/          <- create this
  CM_RO.v1.mod <- .mod files go here
  CM_FS.v1.mod
CourseOfTemptation.html
```

Step 3: Add your mods

Place your `.mod` files inside the `mods/` folder. The patcher loads every `.mod` file it finds there. File order within the folder does not matter -- load order is determined by the dependency graph (`cm_dependencies.json`) if present, or by filename alphabetically if not.

Step 4: Run the patcher

Double-click `KittyPatcher.py` (if Python is registered for `.py` files) or run it from a terminal:

```
python KittyPatcher.py
```

On first run, the patcher will ask you to select a browser. Type the number next to your browser and press Enter. Your choice is saved to `kitty_config.toml` and used automatically on every future run.

Step 5: Verify it worked

The patcher opens the game in your browser automatically. To confirm mods applied correctly, check the `logs/` folder:

```
logs/
  FailsPatchLog.txt <- should be empty if all mods applied
  MainPatchLog.txt  <- shows which mods loaded and how many directives ran
```

NOTE: An empty `FailsPatchLog.txt` means all directives from all mods applied successfully. If it contains entries, see the [Troubleshooting](#) section.

Browser Configuration

On first run the patcher prompts you to choose a browser. To change your selection later, open `kitty_config.toml` and edit the browser line:

```
[browser]
browser = "chrome"    # options: chrome, firefox, edge, safari, default
```

Setting browser to "default" uses your operating system's default browser. On Windows you can also specify a full path:

```
[browser]
browser = "C:/Program Files/Google/Chrome/Application/chrome.exe"
```

kitty_config.toml: Complete Reference

KittyPatcher reads its configuration from `kitty_config.toml` in the same directory as the patcher. All settings are optional -- defaults are shown below.

[browser]

```
[browser]
browser = "default"    # browser to open game in after patching
```

[logging]

```
[logging]
verbose = false    # set to true for detailed Replace:/With: logging
```

Verbose mode writes the full old and new content of every Replace: operation to `ModPatchLog.txt`. Useful when a mod is not behaving as expected.

[cache]

```
[cache]
enabled = true      # set to false to re-patch from scratch every run
max_entries = 10    # number of cached patch results to keep
```

The cache stores the result of a successful patch run keyed by a hash of the vanilla HTML and all .mod files. If nothing has changed since the last run, the cached result is used directly -- the game opens in about 0.2 seconds instead of several minutes.

Increase `max_entries` if you frequently switch between mod configurations. Each unique combination of mods is a separate cache entry.

NOTE: Delete the `cache/` folder at any time to force a full re-patch. The cache is never used when there are patch failures -- only clean runs are cached.

Managing Mods

Enabling and disabling mods

To disable a mod without deleting it, rename the file so it does not end in `.mod` -- for example, rename `CM_MyMod.v1.mod` to `CM_MyMod.v1.mod.disabled`. The patcher only loads files ending in `.mod`, `.kdiff`, or `.patch`.

Mod load order

If `cm_dependencies.json` is present alongside the patcher, mods load in dependency order -- parent mods always before child mods. Within the same dependency level, the `Priority:` field in each mod's header controls ordering (lower number = loads earlier, default 1000). Without `cm_dependencies.json`, mods load in alphabetical filename order.

```
# cm_dependencies.json example:
{
  "CM_FS.v1":    ["CM_FM.v1"],
  "CM_CE.v1":    ["CM_FM.v1"],
  "CM_CE_FS.v1": ["CM_CE.v1", "CM_FS.v1"]
}
```

This tells the patcher that CM_FS and CM_CE both depend on CM_FM (so CM_FM loads first), and CM_CE_FS depends on both (so it loads last). If you received cm_dependencies.json with your mod pack, place it next to KittyPatcher.py and it is used automatically.

Checking which mods loaded

Open logs/MainPatchLog.txt after a patch run. It lists every mod that was found, loaded, and how many directives it applied:

```
Mods loaded (4):
  CM_FM.v1.mod      -- 12 directives applied, 0 failed
  CM_RO.v1.mod      -- 28 directives applied, 0 failed
  CM_FS.v1.mod      -- 19 directives applied, 0 failed
  CM_CE_FS.v1.mod   -- 7 directives applied, 0 failed
Total: 66 applied, 0 failed
Patch time: 3.8s
```

Troubleshooting

The game does not open after patching

Check whether CourseOfTemptation.patched.html was created in the same folder as the original HTML. If it was not created, look in the terminal for an error message. Common causes:

- Python is not installed or not in the system PATH.

- The patcher cannot find CourseOfTemptation.html -- try running with the full path: python KittyPatcher.py /path/to/CourseOfTemptation.html

- A mod caused a critical error during patching. Check MainPatchLog.txt for the specific error.

FailsPatchLog.txt is not empty

One or more directive anchors could not be found in the game HTML. This usually means the mod was written for a different version of the game than the one you have. The game still launches -- failed directives are skipped, not fatal. Options:

- Check if a newer version of the mod exists that targets your game version.

- Contact the mod author with the contents of FailsPatchLog.txt.

- If the mod was previously working and stopped after a game update, the game changed the code the mod was patching. A mod update is needed.

A mod's content is not appearing in game

Even if FailsPatchLog.txt is empty, a mod might not work correctly if:

- Another mod it depends on is not loaded. Check that all required mods are in the mods/ folder.

- The mod requires a specific in-game condition to trigger (correct location, time of day, relationship state, etc.).

- The cache is serving a stale result. Delete the cache/ folder and re-run the patcher to force a fresh patch.

Two mods are conflicting

Open FailsPatchLog.txt and look for lines that say CONFLICT DETECTED. This means two mods are trying to modify the same code. The last mod in load order wins. Options:

- Check if either mod has a known incompatibility with the other.

- Adjust the Priority: or cm_dependencies.json order so the preferred mod applies last.

- Contact the authors -- a compatibility submod may already exist.

The patcher is very slow on every run

If startup is slow even when no mods have changed, check that the cache is enabled in kitty_config.toml:

```
[cache]
enabled = true
```

Also verify the cache/ folder was not accidentally deleted or excluded from the directory. A clean-run patch with many mods can take several minutes -- subsequent runs with the cache enabled take under a second.

The game won't load in my browser

The patched HTML is a large file (16MB+). Some browsers load it slowly. If the game loads but is blank or shows an error:

- Try a different browser (Chrome and Firefox handle large HTML files best).

- Check the browser console (F12) for JavaScript errors -- a mod may have injected code with a syntax error.

- Disable mods one at a time to identify which mod causes the issue.

Frequently Asked Questions

Does KittyPatcher modify the original game file?

No. KittyPatcher always works from a backup copy of the original CourseOfTemptation.html. On the first run it saves a backup to the backup/ folder. Every subsequent run restores from that backup before applying mods. The original file is never modified.

Do I need to re-run the patcher after every game update?

Yes. When you download a new version of the game, replace CourseOfTemptation.html in the game folder and run the patcher again. The patcher will detect the new file (different hash), invalidate the cache, and re-patch from scratch. Some mods may fail if the game update changed the code they target -- check FailsPatchLog.txt.

Can I run the patcher more than once?

Yes, safely. Every run starts by restoring the original HTML from backup, so running twice produces the same result as running once. You cannot "stack" patches by running multiple times.

Can I add or remove mods without starting a new game?

Yes. Adding or removing a .mod file and re-patching changes which mods are active. Whether that affects a save in progress depends on the specific mod -- mods that add new story variables or passages generally work fine mid-playthrough. Check the mod's documentation.

What is the cache/ folder?

The cache stores the result of a successful patch run. If you run the patcher again with the same mods and the same game HTML, it serves the cached result instead of re-patching. This makes subsequent runs nearly instant. The cache is automatically invalidated when any .mod file or the game HTML changes.

Why does my game say "Page not found" or similar?

This usually means the browser is trying to open the file from a URL that doesn't exist. Make sure KittyPatcher opened the file directly (as a local file:// URL) rather than a web URL. If the browser is opening a bookmark instead, clear it and let the patcher open the file.

What does the Priority: field do?

When multiple mods are at the same level in the dependency graph, Priority: controls their relative load order. Lower numbers load earlier. The default is 1000. A mod with Priority: 500 loads before one with Priority: 1000 at the same dependency level. This rarely matters unless two mods modify the same passage and you want to control which modification applies last.

Handling Game Updates

When a new version of Course of Temptation is released, follow these steps:

Step 1: Back up your current setup

Before doing anything, copy your current mods/ folder somewhere safe. If the new game version breaks some mods, you can revert to the old game version and your saved mods folder.

Step 2: Replace the game HTML

Download the new game version and replace CourseOfTemptation.html in your game folder. Delete the backup/ folder so the patcher creates a fresh backup from the new version.

WARNING: Do not replace CourseOfTemptation.html with the patched version. Always use the original unmodified HTML from the game download. The patched version is in the same folder as CourseOfTemptation.patched.html.

Step 3: Run the patcher and check the logs

Run KittyPatcher.py. Even if the game opens successfully, check FailsPatchLog.txt for failures. A new game version may have changed code that some mods target, causing those directives to fail silently.

Step 4: Identify broken mods

FailsPatchLog.txt entries that say "No match found" indicate directives that no longer find their target in the new game HTML. Check whether updated versions of those mods are available. If not, the mod needs to be updated by its author to target the new game version.

Step 5: Disable broken mods temporarily

If a mod is broken and no update is available, rename it to disable it (e.g., CM_MyMod.v1.mod.disabled) and continue playing with the other mods until an update is released. Report the failure to the mod author with the relevant lines from FailsPatchLog.txt.

Quick Reference: All Directive Syntax

```
// TIER S -- additive, zero porting cost
Add Passage: PassageName
tags: tag1 tag2
[passage body]
Add Passage: <tw-passagedata pid="auto" name="Name" tags="" position="0,0" size="100,100">
passage content here
</tw-passagedata>
Add Javascript: // no duplicate check -- guard stateful init
Add CSS:        // /* */ safe; // NOT valid in CSS
Add Events: { passage: "...", tags: [...], chance: 0.05, conditions: [] },
Add Function [setup.myFunc]: setup.myFunc = function(arg) { return arg; };
Add Variable [myVar]: true
Rename Passage [OldName]: NewName
Move Passage [OldName] To [NewName]:
Add Tag To Passage [Name]: newtag
Remove Tag From Passage [Name]: oldtag

// TIER 1 -- passage name anchor
Prepend To Passage [Name]: With: content at start
Append To Passage [Name]: With: content at end

// TIER 2 -- near-zero porting cost
Hook [setup.existingFunc]: before: | after: | around: (v0.7.0)
Insert Before [anchorString]: With: code before
Insert After [anchorString]: With: code after

// TIER 2.5 -- scoped
Insert Into Function [setup.func]: anchor With: code after anchor
Replace In Function [setup.func]: anchor With: replacement
Delete In Passage [PassageName]: text to delete

// TIER 3
Replace Function [setup.existingFunc]: // complete body, no braces
Replace Function Signature [setup.func]: newParam1, newParam2

// TIER 4-5
Replace In Passage [PassageName]: anchor With: replacement
Delete Span [start] To [end]:
ReplaceReg [(?s)prefix(CAPTURE)suffix]: With{1}: replacement
Replace: <exact HTML> With: [raw Twine -- auto-escaped in v0.7.0]

// STRUCTURAL
Append Array [arrayName In Function [setup.init]]: "new item",
Append Variable to Class [ClassName]: this.newProp = 0;
Append Function to Class [ClassName]: newMethod() { return this.newProp; }
Insert Into Array [arrayName]: "item",
Insert Into Object [objectName]: key: value,
```

Delete Block [PassageName]:

Guard Quick Reference (v0.7.0: 12 guards)

IfExists [string]:	IfNotExists [string]:
IfRegExists [pattern]:	IfRegNotExists [pattern]:
IfModLoaded [file.mod]:	IfModNotLoaded [file.mod]:
IfPassageExists [Name]:	IfPassageNotExists [Name]:
IfFunctionExists [name]:	IfFunctionNotExists [name]:
IfPassageContains [Name text]:	IfPassageHasTag [Name tag]: <- v0.7.0

Log Files at a Glance

File	Contents
ModPatchLog.txt	Every attempt, success, skip, and failure with the search key
FailsPatchLog.txt	Only failures and conflict warnings
MainPatchLog.txt	Summary: counts, successful mods, failed mods, timing
CustomLog.txt	Custom mod messages

Workflow Reference

A typical modding workflow with KittyPatcher v0.7.4:

Step	Tool	What happens
1. Author	Text editor	.mod file written per this guide conventions
2. Validate	KittyValidate.py	Every directive anchor checked against game HTML. Zero failures required before continuing.
3. Patch	KittyPatcher.py	All .mod files applied in Phase 0-7 order. patched.html generated. Cache written on clean run.
4. Test	Browser + game	Launch game, test mod content, verify no runtime errors.
5. Debug	ModPatchLog.txt	Check logs for skipped or failed directives. Re-author if needed.